

MLP vs Gate Engine: Compatibility & Integration Strategies

Current State Analysis

Your Existing MLP Model

```
python

# From your notebook:
Threshold: 0.9988444286248084 # ≈ 99.88%
Features: 16 (Tag, t_min, Triage, etc.)
Problem: Essentially never predicts positive → useless for operations
```

Root Issue: Your MLP was trained to predict "admission timing" which is inherently unpredictable due to:

- Bed availability fluctuations
- Social factors (nursing home placement)
- Institutional politics (night shift resistance)
- External factors (family availability, insurance)

Gate Engine Design

```
python

# Deterministic workflow completion tracking
Purpose: Assess "readiness for next step" not "probability of admission"
Output: Actionable priorities, specific next steps, ETAs
Method: Rule-based logic reflecting operational reality
```

3 Integration Architecture Options

Option 1: Pure Replacement (Recommended)

16 Features

+ Context

→

Gate Engine

(Rules)

→

Operational

Decisions

✗ Remove: MLP prediction

✓ Keep: Feature extraction, data pipeline, event logging

✓ Add: Gate engine, rule-based assessment

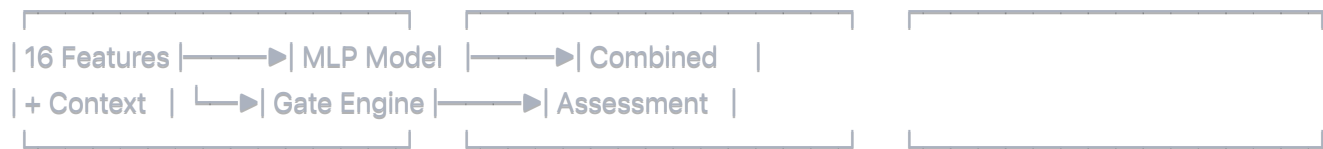
Pros:

- Eliminates useless MLP predictions
- Provides actionable operational intelligence
- Deterministic and explainable
- Directly addresses your bottlenecks

Cons:

- Loses any potential signal from MLP (though current signal is noise)

Option 2: Hybrid Architecture



Gate Engine = Primary decision maker
MLP = Additional risk signal (if retrained)

Implementation:

```
python

def hybrid_assessment(features, calculators, context):
    # Gate engine (primary)
    gate_result = gate_engine.compute_overall_readiness(features, calculators, context)

    # MLP (supplementary, if retrained with better threshold)
    try:
        ml_result = predict_one(features)
        # Only use ML if it provides meaningful signal
        if ml_result['p'] > 0.1: # More reasonable threshold
            gate_result['ml_risk_flag'] = True
            gate_result['priority'] += 1 # Bump priority slightly
    except:
        pass

    return gate_result
```

Pros:

- Preserves investment in ML infrastructure
- Gate engine handles operational logic
- ML could add clinical risk nuance (if retrained)

Cons:

- More complex architecture
- Current MLP adds no value
- Risk of conflicting signals

Option 3: Staged Architecture

Stage 1: Gate Engine (Operational Readiness)

- └ Ready for discharge? → Social/timing gates
- └ Ready for transfer? → Medical completion gates
- └ Ready for ICU? → Resource/safety gates

Stage 2: MLP (Clinical Risk) - Optional

- └ If operationally ready, assess clinical complexity

Implementation:

python

```
def staged_assessment(features, calculators, context):
    # Stage 1: Operational readiness (Gate Engine)
    gate_result = gate_engine.compute_overall_readiness(features, calculators, context)

    # Stage 2: Clinical complexity (MLP) - only if operationally ready
    if gate_result['overall_readiness'] > 0.7:
        ml_result = predict_one(features)
        # Use ML to refine priority among "ready" patients
        gate_result['clinical_complexity'] = ml_result['p']

    return gate_result
```

Recommendation: Option 1 (Pure Replacement)

Why Replace, Not Combine?

1. Your MLP is Fundamentally Broken

- Threshold 99.88% means it never triggers
- Trained on wrong target (admission timing vs workflow readiness)
- No clinical value in current form

2. Different Problem Domains

MLP Problem: "Will this patient be admitted?" (probabilistic)

Gate Problem: "Is this patient ready for next step?" (deterministic)

These are orthogonal questions requiring different approaches

3. Operational Focus Needed

- Your bottlenecks are operational (bed managers, night shift, social)
- Not clinical risk prediction
- Rules-based logic maps directly to your constraints

4. Explainability Requirements

- Clinical staff need to understand WHY a patient has high priority
- "Troponin delta pending" is actionable
- "MLP score 0.23" is not actionable

Migration Path

Phase 1: Parallel Operation

```
python

# Keep both during transition
ml_result = predict_one(features) # Current system
gate_result = gate_engine.compute_overall_readiness(features, calculators, context)

# Log both for comparison
_append_event({
    "type": "parallel_assessment",
    "ml_prediction": ml_result['y'],
    "gate_priority": gate_result['priority'],
    "gate_actions": gate_result['next_action']
})
```

Phase 2: Gate Engine Primary

```
python
```

```
# Use gate engine for operational decisions  
# Keep ML logging for potential future retraining  
result = gate_engine.compute_overall_readiness(features, calculators, context)
```

Phase 3: Pure Gate Engine

```
python
```

```
# Remove MLP entirely, keep infrastructure  
# Gate engine becomes single source of truth
```

Implementation Code Structure

Compatible with Your Existing Infrastructure

```
python
```

```
# Your existing code (unchanged):  
def score_and_log(row: dict):  
    res = predict_one(row) # MLP  
    _append_event({"type": "ml_score", "res": res})  
    return res  
  
# New gate-aware version:  
def score_and_gate(row: dict, calculators=None, context=None):  
    # Keep ML for comparison (optional)  
    ml_res = predict_one(row) if CONFIG.get("INCLUDE_ML") else None  
  
    # Gate engine assessment  
    gate_res = gate_engine.compute_overall_readiness(row, calculators, context)  
  
    # Log both  
    _append_event({  
        "type": "hybrid_score",  
        "ml_result": ml_res,  
        "gate_result": gate_res  
    })  
  
    return gate_res # Use gate engine for decisions
```

DataFrame Integration (Replaces Gate_pos)

python

Before: useless Gate_pos column

```
df['Gate_pos'] = [predict_one(row)['y'] for row in df.iterrows()] # Always False
```

After: actionable gate columns

```
df_enhanced = enhance_dataframe_with_gates(df, calculator_df)
```

New columns: priority, gates, next_action, readiness_score, eta_minutes

Bottom Line: Architecture Recommendation

Replace the MLP with Gate Engine because:

1. **MLP solves wrong problem** (admission prediction vs workflow readiness)
2. **MLP is broken** (99.88% threshold = never triggers)
3. **Gate Engine addresses real bottlenecks** (your UKE operational knowledge)
4. **Same data pipeline** (16 features + calculators work with both)
5. **Better operational outcomes** (actionable priorities vs useless probabilities)

The gate engine IS compatible with your existing infrastructure - it uses the same 16 features, integrates with WorkflowState, respects your CONFIG guards, and follows your event logging patterns.

You get to keep all your clinical calculators, data pipeline, and UI - just replace the broken prediction model with actionable operational intelligence.